Gestion des livraisons

Rapport : choix algorithmiques et analyse de complexité

1. Langage et organisation

Nous avons choisi Python 3. Il permet d'écrire les algorithmes de façon courte et lisible, et son dictionnaire (dict) est une table de hachage. Python est plus lent que le C, donc nous avons surtout cherché à réduire la complexité des algorithmes. Seule la bibliothèque standard est utilisée, et les tris et le sac à dos sont codés à la main.

Le code est réparti en modules : colis.py (le colis), catalogue.py (stockage, tris, statistiques), tri.py (algorithmes de tri), chargement.py (chargement du camion), main.py (menu), benchmark.py (mesures de temps) et tests.py (tests).

Notations : n = nombre de colis, C = capacité du camion en kg, n' = nombre de colis gardés après la réduction (section 4). La priorité va de 1 (faible) à 5 (urgent).

2. Choix n°1 : stocker et rechercher les colis

Problème	Ajouter des colis et retrouver vite un colis par son identifiant, même avec 100 000 colis.
Solution	Une table de hachage (dict) : identifiant → colis. L'identifiant donne directement l'emplacement du colis, sans parcourir la liste. Un identifiant déjà utilisé est refusé.
Temps	O(1) en moyenne pour ajouter, chercher ou supprimer.
Espace	O(n).
Alternative	Liste simple : recherche en O(n). Liste triée : recherche en O(log n) mais ajout en O(n). Arbre équilibré : tout en O(log n).
Justification	C'est la seule structure où l'ajout et la recherche sont tous les deux en temps constant. Mesure : 1 000 recherches parmi 100 000 colis prennent 0,4 ms, contre 3 100 ms avec une liste simple.

3. Choix n°2 : trier les colis (poids, valeur, priorité)

Problème	Afficher les colis triés par poids, par valeur ou par priorité.
Solution	Poids et valeur : tri fusion. On coupe la liste en petits morceaux triés, puis on les fusionne deux par deux. Priorité : tri par dénombrement. Comme il n'y a que 5 priorités, on range chaque colis dans l'une des 5 « cases », puis on les met bout à bout. Le résultat est gardé en mémoire tant que la liste ne change pas, et l'affichage se fait par pages de 20 lignes.
Temps	Tri fusion : O(n log n), même dans le pire cas. Dénombrement : O(n).
Espace	O(n).
Alternative	Tri rapide : souvent rapide mais $O(n^2)$ dans le pire cas. Tri par insertion : $O(n^2)$, trop lent pour 100 000 colis. sorted() de Python : plus rapide, mais pas codé par nous.
Justification	Le tri fusion est toujours en O(n log n) et il garde l'ordre d'arrivée des colis égaux (tri stable). Pour la priorité, le dénombrement est 45 fois plus rapide (12 ms contre 534 ms pour 100 000 colis).

4. Choix n°3 : charger le camion

Il faut choisir les colis qui rapportent la plus grande valeur totale sans dépasser le poids maximal C. C'est le problème du « sac à dos ». Essayer toutes les combinaisons est impossible : avec 100 colis, il y en a environ 10^{30}.

Programmation dynamique (DP)

On remplit un tableau dp où dp[c] est la meilleure valeur possible pour un poids maximal c. On ajoute les colis un par un, et pour chaque capacité on garde le meilleur des deux choix :

```
dp[c] = max( dp[c] ,          dp[c - poids] + valeur )
             (sans le colis)  (avec le colis)
```

Comme tous les cas sont examinés, le résultat est toujours optimal. Nous l'avons vérifié sur 150 exemples en le comparant à l'essai de toutes les combinaisons.

Exemple : colis A (8 kg, 20 €), B (6 kg, 25 €), C (5 kg, 18 €), D (4 kg, 14 €), camion de 9 kg. Un algorithme « glouton », qui prend d'abord le colis le plus rentable au kg, choisit B et ne peut plus rien ajouter : 25 €. La DP trouve C + D : 32 €.

Réduction pour aller plus vite

Un camion de C kg ne peut pas contenir plus de C / w colis de poids w. Pour chaque poids, on garde donc seulement les C / w colis les plus chers et on ignore les autres. Le résultat ne change pas, mais il reste beaucoup moins de colis : 100 000 colis deviennent 4 479 pour un camion de 1 000 kg.

Problème	Trouver le chargement de plus grande valeur sans dépasser C kg.
Solution	1) Réduction : garder les C / w meilleurs colis de chaque poids w. 2) Programmation dynamique sur les colis restants. 3) Si le calcul est trop gros, on passe au glouton et on prévient l'utilisateur que la solution est approchée.
Temps	$O(n \log n + n' \cdot C)$.
Espace	$O(n' \cdot C)$ (environ 4,5 Mo pour 4 479 colis et C = 1 000).
Alternative	Toutes les combinaisons : $O(2^n)$, impossible au-delà de 25 colis. Glouton : rapide mais pas toujours le meilleur résultat.
Justification	Le sujet demande la meilleure valeur possible, il faut donc une méthode exacte. Le poids d'un camion reste raisonnable (environ 1 000 kg), donc la DP est assez rapide : 1,2 s pour 100 000 colis.

5. Choix n°4 : tenir compte de la priorité

On donne plus de valeur aux colis urgents, puis on utilise la même méthode de chargement :

```
valeur_pondérée = valeur × (1 + α × (priorité − 1) / 4)
```

Un colis de priorité 1 garde sa valeur. Un colis de priorité 5 voit sa valeur multipliée par (1 + α). L'utilisateur choisit α : avec α = 0 la priorité est ignorée, avec α = 1 (par défaut) un colis urgent compte double. Le poids n'entre pas dans ce calcul, car il est déjà pris en compte par la limite du camion.

Exemple avec un camion de 15 kg (A est urgent, priorité 5) :

α	Chargement choisi	Poids	Valeur réelle
0	B + C + D	15 kg	57 €
1	B + C + D	15 kg	57 €
3	A + C	13 kg	38 €

Plus α est grand, plus le colis urgent A est chargé, même si la valeur totale baisse.

Problème	Favoriser les colis urgents tout en gardant une bonne valeur totale.
Solution	Multiplier la valeur par un coefficient lié à la priorité, puis lancer la DP.
Temps	O(n) pour les nouvelles valeurs, puis le même coût que le chargement.
Espace	O(n).
Alternative	Charger d'abord tous les colis de priorité 5 : simple, mais un colis urgent sans valeur peut prendre la place de colis bien plus utiles.
Justification	La règle est simple à expliquer, réglable avec α, et le résultat reste optimal pour ce score.

6. Passage à l'échelle

Mesures faites avec benchmark.py (temps en millisecondes) :

n	1 000 recherches (hachage)	1 000 recherches (liste)	Tri fusion	Dénombrement	Chargement (DP)
100	0,10	2,4	0,17	0,01	16
1 000	0,12	25	2,3	0,10	113
10 000	0,11	289	34	1,0	377
100 000	0,39	3 101	534	12	1 232

Recherche : avec la table de hachage, le temps reste presque le même quel que soit n. Avec une liste, il augmente autant que n.

Tri : le temps augmente un peu plus vite que n, comme prévu pour O(n log n). Une demi-seconde pour 100 000 colis reste acceptable.

Chargement : grâce à la réduction, le temps augmente peu avec n. Il dépend surtout de la capacité C du camion. Pour aller plus vite, on pourrait arrondir les poids (par exemple par 5 kg).

7. Conclusion

Opération	Méthode	Temps
Ajouter / rechercher	Table de hachage	O(1)
Trier par poids / valeur	Tri fusion	O(n log n)
Trier par priorité	Tri par dénombrement	O(n)
Charger le camion	Réduction + programmation dynamique	O(n log n + n'·C)
Priorité	Valeur pondérée	O(n)

Chaque structure a été choisie pour rendre rapide l'opération dont elle s'occupe. Le chargement, qui est la partie difficile, donne toujours le meilleur résultat. Les mesures jusqu'à 100 000 colis confirment les complexités annoncées.